

LAPORAN JOBSHEET 20: DEPLOYVERCEL MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

PRAKTIKUM 1 – MEMBUAT REPOSITORY GITHUB

1. Buat Repository Baru

1. Login ke GitHub
2. Klik New Repository
3. Beri nama repository
4. Pilih Private/Public
5. Klik Create Repository

2. Hubungkan Project Lokal ke GitHub

- Cek konfigurasi Git:

```
git config --global user.name git config --global user.email
```

- Jika belum ada:

```
git config --global user.name "username_github" git config --global user.email "email_github"
```

3. Tambahkan Remote Repository

- `git remote add origin https://github.com/username/repository.git`

```
git add .
```

```
git commit -m "Initial deployment"
```

```
git push origin main
```

- Pastikan repository di GitHub sudah terisi file project.

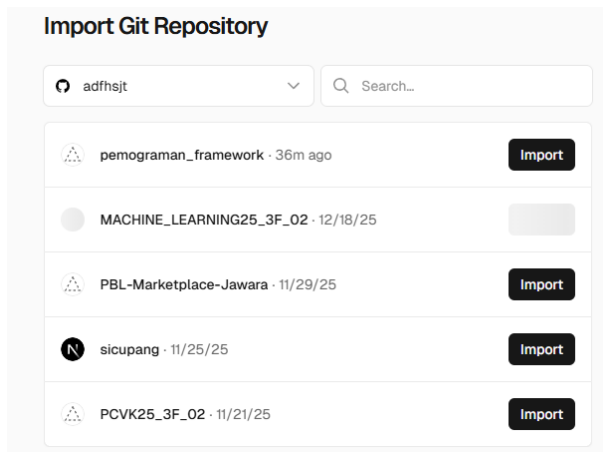
PRAKTIKUM 2 – DEPLOYMENT KE VERCEL

Login ke Vercel

- Buka: <https://vercel.com>
- Login menggunakan GitHub (kalau blm punya akun buat terlebih dahulu)

Import Project

1. Klik Add New Project
2. Install terlebih dahulu githubnya
3. Klik Import



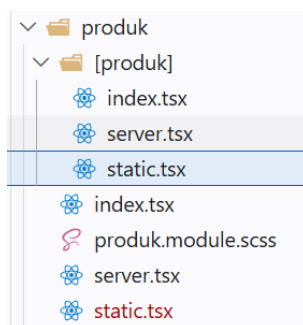
Note : sebelum diimport lakukan konfigurasi terlebih dahulu

C. MENGATASI ERROR SAAT DEPLOYMENT

- Dikarenakan pada code masih terdapat code static-site generation

Masalah: Static Site Generation Gagal

- Hapus file static.tsx



- Comment pada line 46 pada file [produk].tsx yang berhubungan dengan static-site generation dicomment semua

```

pemograman_framework - static.tsx
1 // import DetailProduk from ".././../views/DetailProduct";
2 // import { ProductType } from "../types/Product.type";
3
4 // const HalamanProdukStatic = ({ product }: { product: ProductType }) => {
5 //   return (
6 //     <>
7 //       <div className="text-2xl font-bold ml-4">Detail Produk Static</div>
8 //       <div>
9 //         <DetailProduk products={product} />
10 //       </div>
11 //     </>
12 //   );
13 // };
14
15 // export default HalamanProdukStatic;
16
17 // { /Digunakan static-side generation/ }
18 // export async function getStaticPaths() {
19 //   const res = await fetch('http://localhost:3000/api/products');
20 //   const response = await res.json();
21 //   const paths = response.data.map((product: ProductType) => ({
22 //     params: { produk: product.id },
23 //   }));
24 //   return {
25 //     paths,
26 //     fallback: false,
27 //   };
28 // }
29 //
30 //
31 //
32 //
33 // export async function getStaticProps({ params }: { params: { produk: string } }) {
34 //   const res = await fetch('http://localhost:3000/api/products/${params.produk}');
35 //   const response = await res.json();
36 //   const response = { data: ProductType } = await res.json();
37 //   console.log("Data produk yang diambil dari API:", response);
38 //   return {
39 //     props: {
40 //       product: response.data,
41 //     },
42 //   };
43 // }
44 //

```

Solusi

Gunakan SSR (Server Side Rendering)

- SSR yang sebelumnya di comment dibuka commentnya pada file [produk].tsx

```

pemograman_framework - index.tsx
1 import DetailProduk from ".././../views/DetailProduct";
2 import { ProductType } from "../types/Product.type";
3
4 // const HalamanProdukServer = ({ product }: { product: ProductType }) => {
5 //   return (
6 //     <>
7 //       <div className="text-2xl font-bold ml-4">Detail Produk Server</div>
8 //       <div>
9 //         <DetailProduk products={product} />
10 //       </div>
11 //     </>
12 //   );
13 // };
14
15 // export default HalamanProdukServer;
16
17 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
18 // { /Digunakan server-side rendering/ }
19 // export async function getServerSideProps({ params }: { params: { produk: string } }) {
20 //   const res = await fetch('http://localhost:3000/api/products/${params.produk}');
21 //   const response = await res.json();
22 //   console.log("Data produk yang diambil dari API:", response);
23 //   return {
24 //     props: {
25 //       product: response.data, //Pastikan untuk memberikan nilai default jika data tidak tersedia
26 //     },
27 //   };
28 // }
29 //

```

```

pemograman_framework - index.tsx
1 import TemplatProduk from ".././../views/Produk";
2 import { ProductType } from "../types/Product.type";
3
4 // const HalamanProdukServer = ({ products }: { products: ProductType[] }) => {
5 //   const { products } = props;
6 //   return (
7 //     <div>
8 //       <div className="font-bold text-3xl pl-4">Halaman Produk Server</div>
9 //       <TemplatProduk products={products} />
10 //     </div>
11 //   );
12 // };
13 // export default HalamanProdukServer;
14
15 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
16 // export async function getServerSideProps() {
17 //   const res = await fetch('http://localhost:3000/api/products');
18 //   const response = await res.json();
19 //   console.log("Data produk yang diambil dari API:", response);
20 //   return {
21 //     props: {
22 //       products: response.data,
23 //     },
24 //   };
25 // }
26 //

```

Gunakan Environment Variable

- Buat di .env.local:

o NEXT_PUBLIC_API_URL=http://localhost:3000

```
pemograman_framework - .env.local
12 NEXT_PUBLIC_API_URL=http://localhost:3000
```

Ganti semua hardcoded URL:

o process.env.NEXT_PUBLIC_API_URL

o Contoh:

■ `fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/product`)`

o pada file [produk].tsx

```
pemograman_framework - index.tsx
20 const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/api/products/${params?.produk}`);
```

pada file server.tsx

Commit dan push kembali.

Selanjutnya import lakukan pengaturannya sbb

New Project

Importing from GitHub
adhsjt/pemograman_framework master Code/praktikum/my-app

Choose where you want to create the project and give it a name.

Vercel Team: adhsjt's projects Hobby Project Name: pemograman-framework-5jbt

Application Preset: Next.js

Root Directory: Code/praktikum/my-app Edit

Build and Output Settings

Build Command: npm run build or next build

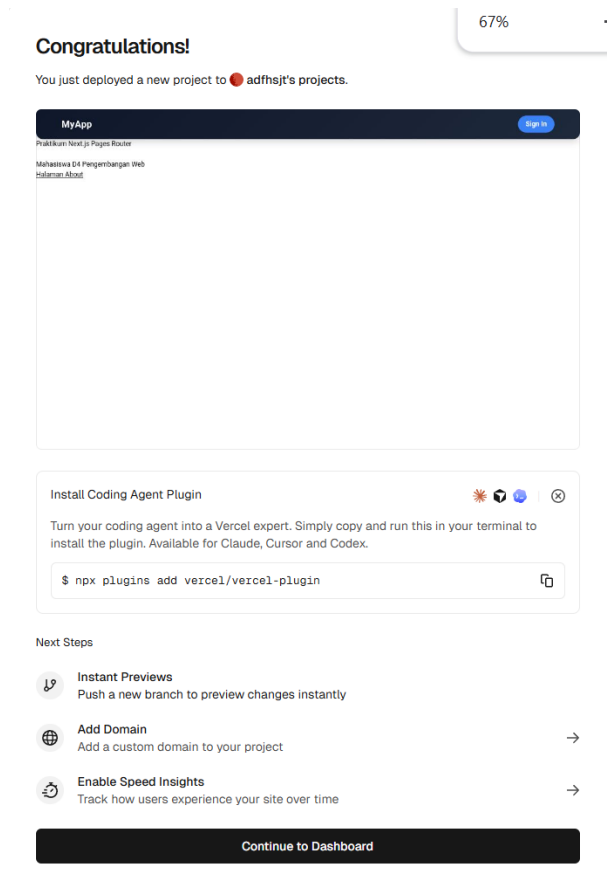
Output Directory: Next.js default

Install Command: npm install --force

Environment Variables

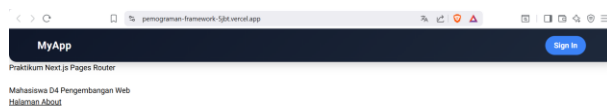
Deploy

Setelah itu klik deploy dan jika berhasil maka akan muncul



Domains +

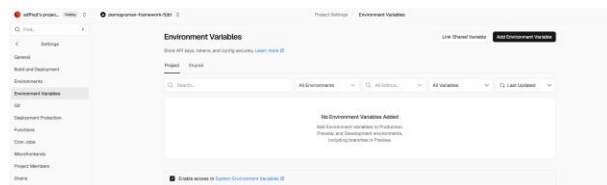
pemograman-framework-5jbt.vercel.app



PRAKTIKUM 3 – MENAMBAHKAN ENVIRONMENT VARIABLE DI VERCEL

Buka Project di Vercel

Settings → Environment Variables

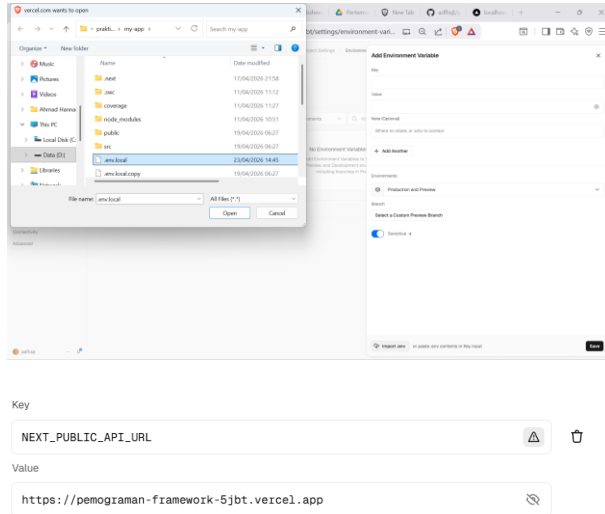


Import dari .env.local

Klik Import .env dan setting next_public_api_url sesuai dengan url vercel project

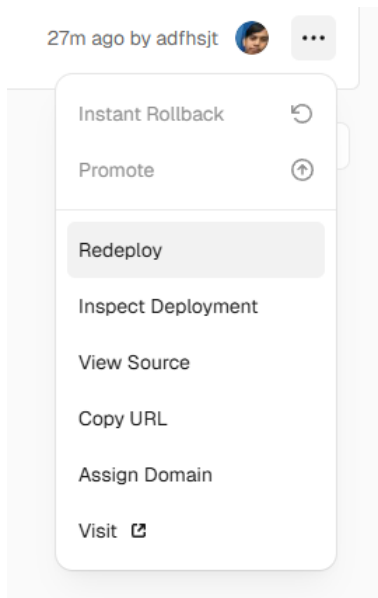
Atau isi manual:

NEXT_PUBLIC_API_URL=https://namaproject.vercel.app



Redeploy

o Deployment → Redeploy



PRAKTIKUM 4 – KONFIGURASI GOOGLE OAUTH PRODUCTION

Masuk ke Google Cloud Console

- Buka google developer console
- Credentials

- Pilih OAuth Client

Authorized JavaScript origins ②

For use with requests from a browser

URIs 1 *
http://localhost:3000

URIs 2 *
https://rain-detection-clothesline-iot.vercel.app

URIs 3 *
https://pemograman-framework-5jbt.vercel.app

+ Add URI

Tambahkan Redirect URI

Authorized redirect URIs ②

For use with requests from a web server

URIs 1 *
http://localhost:3000/api/auth/callback/google

URIs 2 *
https://rain-detection-clothesline-iot.vercel.app/api/auth/callback/goc

URIs 3 *
s://pemograman-framework-5jbt.vercel.app/api/auth/callback/google

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Save

 Cancel

Note : ada kesalahan pada code index.tsx pada folder views/auth/login

Code sebelumnya

```
pemograman_framework - index.tsx
81  {/* Button Login */}
82  <button
83    type="submit"
84    className={style.login__form__item__button}
85    disabled={isLoading}
86  >
87    {isLoading ? "Loading..." : "Login"}
88  </button>
89  <br /><br />
90  {/* Button Login Google */}
91  <button
92    onClick={() => signIn("google", { callbackUrl, redirect: false })}
93    className={style.login__form__item__button}
94    disabled={isLoading}
95    type="button"
96  >
97    {isLoading ? "Loading..." : "sign in with google"}
98  </button>
99  <br /><br />
100  {/* Button Login Github */}
101  <button
102    onClick={() => signIn("github", { callbackUrl, redirect: false })}
103    className={style.login__form__item__button}
104    disabled={isLoading}
105    type="button"
106  >
107    {isLoading ? "Loading..." : "sign in with github"}
108  </button>
```

Code dimodifikasi

```
pemograman_framework - index.tsx

81  {/* Button Login */}
82  <button
83    type="submit"
84    className={style.login__form__item__button}
85    disabled={isLoading}
86  >
87    {isLoading ? "Loading..." : "Login"}
88  </button>
89  <br /><br />
90  {/* Button Login Google */}
91  <button
92    onClick={() => signIn("google", { callbackUrl, redirect: false })}
93    className={style.login__form__item__button}
94    type="button"
95  >
96    Sign in with google
97  </button>
98  <br /><br />
99  {/* Button Login Github */}
100 <button
101   onClick={() => signIn("github", { callbackUrl, redirect: false })}
102   className={style.login__form__item__button}
103   type="button"
104 >
105   Sign in with github
106 </button>
```

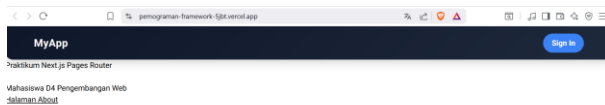
Redeploy Vercel

o Agar environment terbaru terbaca.

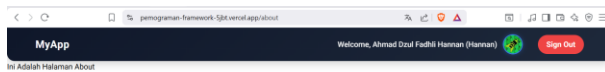
PRAKTIKUM 5 – PENGUJIAN SETELAH DEPLOYMENT

Coba akses:

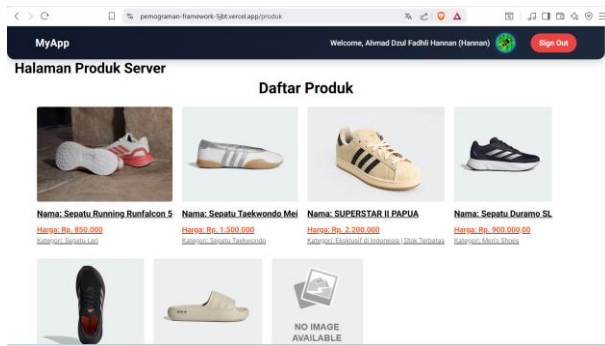
- /



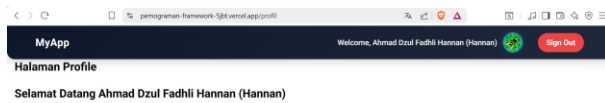
- /about



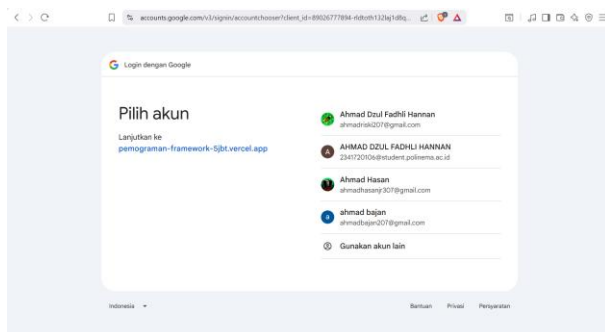
- /product



• /profile

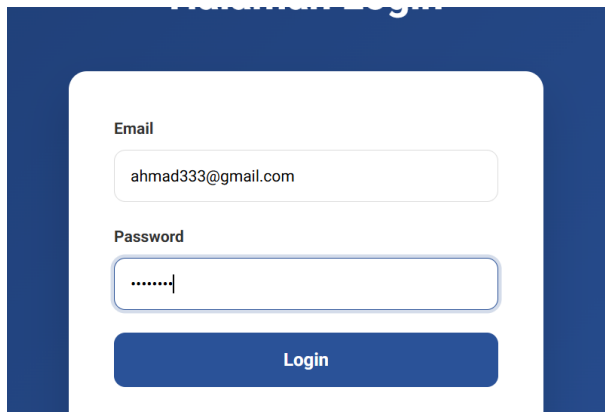


• Login Google



Bisa

• Login credential biasa



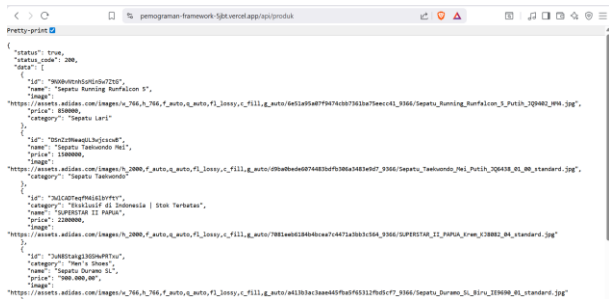


Pastikan:

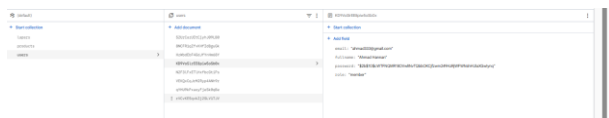
- SSR berjalan

Bisa, akses /produk

- API tidak lagi localhost



- Database terkoneksi



Ini tadi saya register user baru, bisa lihat di bagian pengujian login credential

- Google login berhasil

Bisa, bisa dilihat pada bagian pengujian login google.

ANALISIS KONSEP

Konsep	Penjelasan
SSG	Data diambil saat build
SSR	Data diambil saat request
CSR	Data diambil di browser
Environment Variable	Variabel rahasia/configuration

Redeploy	Build ulang setelah perubahan
OAuth Production	Harus update origin & callback

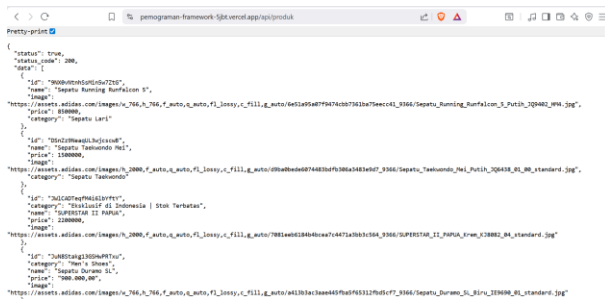
TUGAS PRAKTIKUM

1. Deploy project Next.js ke Vercel

Sudah

2. Pastikan API tidak menggunakan localhost

Sudah



```

{
  "status": true,
  "productId": 200,
  "data": [
    {
      "id": "00000000000000000000000000000000",
      "name": "Sepatu Running Nonfalcon 9",
      "image": "https://assets.adidas.com/images/c_766_f_00000000000000000000000000000000_3166/Sepatu_Running_Nonfalcon_9_Putih_200002.jpg",
      "price": 899999,
      "category": "Sepatu Lari"
    },
    {
      "id": "00000000000000000000000000000000",
      "name": "Sepatu Taekwondo H&I",
      "image": "https://assets.adidas.com/images/c_766_f_00000000000000000000000000000000_3166/Sepatu_Taekwondo_H&I_Putih_200002.jpg",
      "category": "Sepatu Taekwondo"
    },
    {
      "id": "00000000000000000000000000000000",
      "category": "Stasiun di Indonesia | Stok Terbatas",
      "name": "Sepatu Duramo SL",
      "image": "https://assets.adidas.com/images/c_766_f_00000000000000000000000000000000_3166/Sepatu_Duramo_SL_Putih_200002.jpg",
      "price": 229999,
      "category": "Sepatu Lari"
    },
    {
      "id": "00000000000000000000000000000000",
      "category": "Sepatu Taekwondo H&I",
      "name": "Sepatu Taekwondo H&I",
      "image": "https://assets.adidas.com/images/c_766_f_00000000000000000000000000000000_3166/Sepatu_Taekwondo_H&I_Putih_200002.jpg",
      "category": "Sepatu Taekwondo"
    }
  ]
}

```

3. Konfigurasikan Google OAuth production

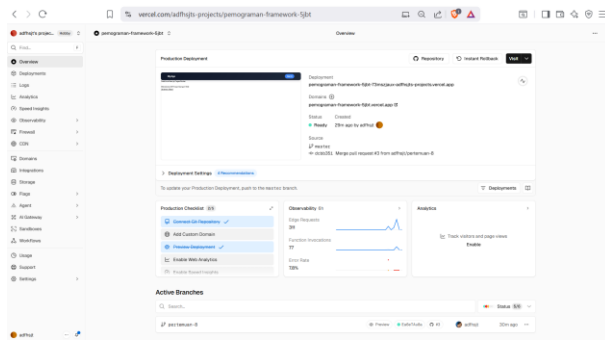
Sudah

4. Lakukan minimal 1 redeploy

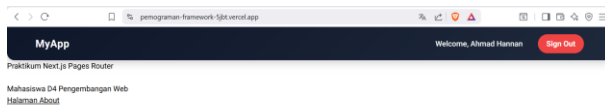
Sudah, 2 kali

5. Dokumentasikan:

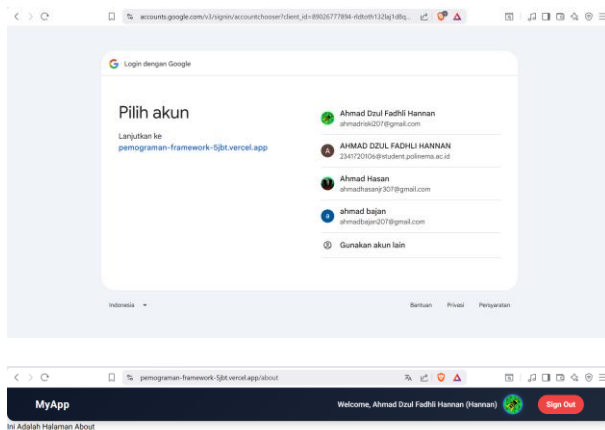
o Screenshot dashboard Vercel



o URL hasil deployment



o Screenshot login Google berhasil'



REFLEKSI & DISKUSI

1. Mengapa localhost tidak boleh digunakan di production?

localhost hanya merujuk pada mesin lokal itu sendiri, sehingga jika digunakan di production, aplikasi tidak akan bisa terhubung ke layanan eksternal atau database yang berada di server lain.

2. Mengapa SSG bisa gagal saat build?

SSG bisa gagal saat build jika terdapat data yang tidak bisa diambil dari API, adanya kesalahan sintaks pada kode, atau variabel environment yang dibutuhkan saat proses fetching belum terpasang.

3. Apa perbedaan SSR dan SSG saat deployment?

Pada saat deployment, SSG menghasilkan file statis yang siap saji sejak awal build, sedangkan SSR membutuhkan server yang aktif untuk merender halaman secara real-time setiap kali ada permintaan dari pengguna.

4. Mengapa perlu redeploy setelah menambahkan environment?

Redeploy diperlukan karena nilai dari variabel environment biasanya diinjeksikan ke dalam kode program pada saat proses build atau inisialisasi awal aplikasi agar dapat terbaca oleh sistem.

5. Apa fungsi redirect URI pada OAuth?

Redirect URI berfungsi sebagai alamat tujuan yang aman bagi penyedia layanan OAuth untuk mengirimkan kembali kode otorisasi atau token kepada aplikasi kamu setelah pengguna berhasil login.

KESIMPULAN

Pada praktikum ini mahasiswa telah:

- ✓ Menghubungkan project dengan GitHub
- ✓ Melakukan deployment ke Vercel
- ✓ Mengelola environment variable
- ✓ Mengatasi error SSG
- ✓ Mengkonfigurasi OAuth production
- ✓ Menguji aplikasi hasil deploy